The Ansh-108 Core - In Plain Words

What it is, what it can do, and how we got here

A project by Ayush - Project ANSH - written 25 June 2026 - **updated 28 June 2026**

*This is the family-and-friends version. No equations you need to follow - just the idea, the honest results, and the road we walked to get them. Every number in here is a real measured or proven result, not a hope. Where something is **not** done yet, it says so plainly. That honesty is the whole point of the project.*

*Update note (28 June): the first edition described the little "108 demonstrator" engine. Since then the full machine was actually built and tested end-to-end - all six of its operations, a clock that "can't lie," and the PC software that feeds it. Those new sections are marked **NEW**.*

---

1. The one-sentence idea

> ***Take the oldest grammar humans ever wrote (Panini's Sanskrit rules, ~2,500 years old) and the newest kind of computer chip, and show they are really the same machine - and then actually build that machine and test it.***

The bridge between the two is a single number you already know from temple malas and prayer beads: **108**.

---

2. Why 108 - explained with no math

108 splits perfectly into **4 x 27**.

Think of it like a lock with two dials - one dial with 4 positions, one with 27. Because 4 and 27 share no common factor, **every single number from 0 to 107 becomes its own unique pair of dial settings.** No two numbers ever land on the same pair. And from the two dial settings you can always rebuild the original number, exactly, with zero guessing.

That "zero guessing" is the magic trick. A normal computer constantly stops to ask *"which case is this? if this, then that..."* - millions of little forks in the road, each one a chance to stall. The Ansh-108 Core **never asks.** It runs the dials side by side, on separate lanes that never wait for each other, and snaps the answer back together at the end. No fork. No guess. No rounding.

The name fits the idea. **Ansh (amsha)** means *a fragment that still carries the whole*. The machine breaks a whole value into honest fragments, carries them in parallel, and makes them whole again with nothing lost.

A note on two numbers (NEW). 108 is the *name and the soul*. But the real working engine uses a bigger special number, **12289**, because it leaves room for proper fingerprints **and** it happens to be the exact number that modern "unbreakable-by-future-computers" security is built on. So: **108 is the identity; 12289 is the workhorse.** Both are real and both were built.

---

3. What it can actually do (the honest list) - NEW, expanded

The full machine now does **six clean operations**, plus it keeps time:

- **Multiply, add, subtract** - the carry-free, no-guessing arithmetic above.
- **Reduce** - tidy a big number back into the dial system.
- **Check if two things are equal (VERIFY)** - a fast yes/no, with no guessing.
- **Make a "fingerprint" (FOLD)** - feed it a pattern (a chant written as short/long beats, a file, a sensor stream) and it crushes the whole thing into

one fixed number. Same input -> same fingerprint, every time. Change one beat, or reorder the beats, and the fingerprint changes completely.

- **Keep an honest clock (NEW).** A counter that only ever ticks **forward**, wraps around after exactly one full cycle (a "Maha Yuga" of 4,320,000 ticks), and that **no command can set, rewind, or fake.** It's a clock that can't lie about *order* - useful for trustworthy timestamps.

Two more honest strengths it has always had:

- **It does hundreds of these at once.** Its real power is width, not single-job speed - many little engines stamping in parallel.
- **It runs perfectly predictably.** Same input -> same output -> same timing, every single time. No jitter. That predictability is rare and valuable (and good for security).

---

4. How we got here - the journey, step by step

We did not just *claim* it works. We took it from an idea all the way to real, routed chip layout and machine-checked proofs - testing at every rung and writing down the result even when it was humbling.

Part A - the little "108 demonstrator" (first edition)

Step	What we did	What we found
The math proof	Brute-force checked the "two dials never collide" claim over all 108 values and all 11,664 combinations.	[OK] Perfect. The trick is real, not poetry.
The chip design (Verilog)	Wrote the true hardware blueprint and simulated it.	[OK] All **11,664** combinations correct, **0 errors**, one stamp every clock tick.
Second opinion	Re-ran the exact blueprint in AMD/Xilinx's own professional simulator.	[OK] Same result - two different tools agree.
Real chip layout	Ran the full professional flow real chip-makers use (on a	[OK] **157.9 MHz**, astonishingly small: **51 logic**

	real Artix-7 chip model).	**cells, 0 multipliers** - about **400 copies** fit on one ~$130 chip.
Machine proof	A proof engine checked correctness for *every* input and timing.	[OK] **3 theorems proven**.

Part B - the full machine, actually built (NEW, 28 June)

Each operation was built and put through the same strict **5-check gate**:

a hand proof in Python, a simulation, a machine proof, a "turn it into real

gates" pass, and a full chip layout. Then they were wired into one chip.

Step	What we did	What we found
The rulebook	Wrote one master program that *defines* correct behaviour; everything else is checked against it.	[OK] 17/17 self-checks.
Add / Subtract	Built and 5-checked them on the workhorse lane.	[OK] All checks green; lay out at **240-260 MHz**.
Equal-check + Fingerprint	Built VERIFY and FOLD. The fingerprint's tidy-up step was checked on **all 268,435,456 possible inputs** - every single one.	[OK] Exact. Fingerprint engine lays out around **41 MHz** (its feedback loop is the slow part - known, and improvable later).
Wire it all together	Built the decoder, the result manager, and the can't-lie clock, into one chip that speaks a simple 32-bit "packet" language.	[OK] Replayed **6,000 mixed commands** with **0 mistakes**; **13 separate properties machine-proven**; whole chip lays out at **42.1 MHz** using only ~4% of a small chip.
The PC software	Built the program that turns a chant text file into the exact packets the chip eats, and reads the answer back.	[OK] Tested over **5,000+ chants** end-to-end, **0 mismatches**; **67/67** software checks pass.

The honest headline: the chip's *math and design are proven*, the whole pipeline

(text file -> chip -> fingerprint) works in simulation, and the only thing left is

loading it onto a physical board.

---

5. The equipment we used

Everything was done on **one laptop** (Windows 11) - no lab, no special hardware:

- **Vivado 2026.1** (AMD/Xilinx) - the same professional chip-design suite the industry uses, in its free mode (final layouts + speed numbers).
- **Icarus Verilog** and **Vivado's own simulator** - two independent simulators, so results were cross-checked, not taken on faith.
- **Yosys** - open-source tool that turns the blueprint into real chip parts.
- **SymbiYosys + math-proof engines (boolector / bitwuzla)** - the formal proof tools that check correctness for *all* inputs, not just examples.
- **Python** - the master "rulebook" model and all the PC software.
- **Target chip:** Xilinx **Artix-7 (xc7a35t)** - the chip on a ~$130 hobbyist board. We designed, simulated, synthesized, and laid it out for this chip.
- **Claude (Anthropic)** as the engineering copilot throughout - writing and running the code, the proofs, and these notes under Ayush's direction.

The full set of files is saved in the `Ansh_108_Core_Artifacts` folder, so the whole thing can be re-run and re-checked by anyone.

---

6. The headline results, in one place

What was tested	How	Result
Two-dial math (no collisions)	brute force	[OK] exact over all 108 values & 11,664 products
Fingerprint tidy-up step	exhaustive check	[OK] correct on **all 268,435,456** inputs
Whole chip, mixed commands	simulation replay	[OK] **6,000 / 0 mistakes**
Whole chip, correctness & timing	machine proof	[OK] **13 properties proven**
108 demonstrator - speed / size	full chip layout	**157.9 MHz**, 51 cells, ~400 copies/chip
Full integrated chip - speed /	full chip layout	**42.1 MHz**, ~4% of a small

size		chip (fingerprint loop is the limiter)
Add / Subtract - speed	full chip layout	**240-260 MHz**
PC software, end-to-end	5,000+ chants	[OK] **0 mismatches**, 67/67 checks

---

7. The clock-watch and the uses - NEW

A standalone "watch" is planned. Because the can't-lie clock already exists inside the chip, it can also become a small **stand-alone time-keeper** - a counter that runs forever, can't be rewound or faked, and tells you where you are in its cycle. The *base* of it is designed; building it is a planned next step.

Honest limit: the chip counts *ticks* perfectly, but turning ticks into exact *seconds* needs a good quartz/atomic reference on the board - "never drifts" is not free. "Can't lie" means the count can't be forged, **not** that the seconds are atomic-perfect.

Where this kind of engine genuinely fits (honest tiers):

- **Strongest:** a building block for **future-proof security** (the number 12289 is the real standard for "safe even against quantum computers"), and for **tamper-evident fingerprints + timestamps** ("this exact data, in this exact order, at this count").
- **Plausible, with more work:** fingerprinting audio/chant patterns; fast "have I seen this exact thing before?" lookups.
- **Speculative (kept honest, not promised):** anything claiming it "replaces a CPU," is "unhackable," or reads cosmic rhythms - those are *not* proven and we don't sell them.

---

8. What it CANNOT do - read this part too

The project's rule is to be as clear about the limits as about the wins.

- **It is NOT a faster calculator than your laptop for one job.** It wins by doing **hundreds at once**, not by being fast at one. It's a *width* engine.
- **It only does add, subtract, multiply, equal-check cleanly.** "Which is bigger?", division, and bit-shuffling need extra steps and lose the magic - so those are deliberately done by the PC, not the chip.
- **One fingerprint is still a smallish number.** A long, truly collision-proof fingerprint needs several of these chained together - the design allows it, but we have not built the long version yet.
- **It is not a password/secret keeper by itself.** A fingerprint proves data wasn't changed; real secrecy needs keys and careful handling on top.
- **"Predictable timing" protects against one kind of leak, not all of them.**
- **No physical chip exists yet.** Everything here is proven in simulation, machine proof, and full layout - exactly how real chips are validated *before* they're manufactured - but it has **not** yet run on a real board. That is the honest current frontier.
- **"Sanskrit grammar = math" is still partly a hypothesis** - real partial support, full claim not proven, and we wrote down exactly where it holds.

---

9. Where it stands today

The first edition had a tiny proof-of-concept engine. **Now the whole machine is built and proven in simulation:** six operations, a clock that can't lie, all wired into one chip and machine-checked, plus the PC software that drives it from a plain text file - tested end-to-end over thousands of chants with zero mistakes. It is **small, perfectly predictable, massively copyable, and honest**

about being a parallel-width engine rather than a single-speed champion.

The next real-world rung is the same as before - loading it onto an actual FPGA board to watch it run in the physical world. Everything is staged and ready for that day.

"The fragment remembers the whole." - the law the whole machine obeys.

- Ayush, Project ANSH